Digital Forensics Practical Report

Steganography Analysis: Embedding, Extraction, and Password Recovery

---

1. Case Information

Field	Details
Case Number	STEG-2026-001
Investigator	Fasinu Temidayo
Date of Analysis	5 September 2026
Lab	ICDFA Laboratory
Carrier Image	`_tower_original_image_for_lab.bmp`
Tools Used	steghide, stegseek, xxd, strings, sha256sum
Role	Junior Digital Forensic Analyst

---

2. Executive Summary

2.1 Investigation Overview

This investigation demonstrates steganography techniques for hiding and extracting data within image files. The lab explores LSB (least significant bit) substitution, embedding/extraction with Steghide, and password recovery using StegSeek — first with a lab-provided example, then repeated independently with a custom payload and password to confirm the method generalizes.

2.2 Key Findings

Task	Result
Carrier Image Hash Recorded	
Secret File Created	
Steghide Embedding	Success
Steghide Extraction	Success
File Integrity (Round 1)	Verified (hashes match)
Password Recovery (Round 1)	Success (`1234` found)
Independent Task (Round 2)	Success (custom note, custom password)
Password Recovery (Round 2)	Success (`temidayo` found)

---

3. Understanding Steganography

3.1 Steganography vs Encryption

Feature	Steganography	Encryption
Purpose	Hide existence of data	Scramble data
Visibility	Hidden from view	Visible as scrambled text
Detection	Difficult to detect	Obvious presence
Implementation	LSB substitution	Mathematical algorithms

3.2 LSB Substitution Explained

What is LSB?

Each pixel in a 24-bit BMP has 3 color channels (R, G, B), each with 8 bits. The LSB is the rightmost bit.

Example:

```
Original: Red = 11111111 (255)
Modified: Red = 11111110 (254) - only the last bit changed
```

The human eye cannot detect the ± 1 color difference.

Why It Works: - Hides 1 bit per color channel (3 bits per pixel) - Visual appearance remains virtually identical - Requires 8 pixels to hide 1 byte of data - Can hide large amounts of data in high-resolution images

---

4. Part A: Evidence Preparation

4.1 Environment Setup

Command:

```
mkdir ~/steganography_lab
cd ~/steganography_lab
mkdir original extracted screenshots wordlists
```

```
┌──(kali㉿kali)-[~]
└─$ mkdir ~/steganography_lab

┌──(kali㉿kali)-[~]
└─$ cd ~/steganography_lab

┌──(kali㉿kali)-[~/steganography_lab]
└─$ mkdir original extracted screenshots wordlists

┌──(kali㉿kali)-[~/steganography_lab]
└─$ sudo apt install -y steghide stegseek xxd exiftool
[sudo] password for kali:
Note, selecting 'libimage-exiftool-perl' instead of 'exiftool'
libimage-exiftool-perl is already the newest version (13.55+dfsg-1).
libimage-exiftool-perl set to manually installed.
The following packages were automatically installed and are no longer required:
  libavc1394-0  libmjpegutils-2.1-0t64  libmpeg2encpp-2.1-0t64  libmplex2-2.1-0t64  libraw23t6
Use 'sudo apt autoremove' to remove them.

Upgrading:
  steghide  xxd

Installing:
  stegseek

Summary:
  Upgrading: 2, Installing: 1, Removing: 0, Not Upgrading: 2177
  Download size: 349 kB
  Space needed: 391 kB / 51.4 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 steghide amd64 0.5.1+git20240220-2 [15
Get:2 http://http.kali.org/kali kali-rolling/main amd64 stegseek amd64 0.6+git20210910.ff677b9
Get:3 http://kali.download/kali kali-rolling/main amd64 xxd amd64 2:9.2.0858-1 [65.2 kB]
Fetched 349 kB in 2s (229 kB/s)
(Reading database ... 429453 files and directories currently installed.)
Preparing to unpack .../steghide_0.5.1+git20240220-2_amd64.deb ...
Unpacking steghide (0.5.1+git20240220-2) over (0.5.1-15) ...
Selecting previously unselected package stegseek.
Preparing to unpack .../stegseek_0.6+git20210910.ff677b9-2_amd64.deb ...
```

Screenshot: working directory

4.2 Download Carrier Image

Command:

```
wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer
```

Result:

```
File: _tower_original_image_for_lab.bmp
Size: 9,277,494 bytes
Type: PC bitmap, Windows 3.x format, 1365 x 2265 x 24
```

```
┌──(kali㉿kali)-[~/steganography_lab]
└─$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Comput
er_Skills_for_Forensics/steganography/_tower_original_image_for_lab.bmp

┌──(kali㉿kali)-[~/steganography_lab]
└─$ ls -la _tower_original_image_for_lab.bmp
-rw-rw-r-- 1 kali kali 9277494 Sep  5 05:33 _tower_original_image_for_lab.bmp

┌──(kali㉿kali)-[~/steganography_lab]
└─$ file _tower_original_image_for_lab.bmp
_tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution
 2834 x 2834 px/m, cbSize 9277494, bits offset 54

┌──(kali㉿kali)-[~/steganography_lab]
└─$ display _tower_original_image_for_lab.bmp
```

Screenshot: file download

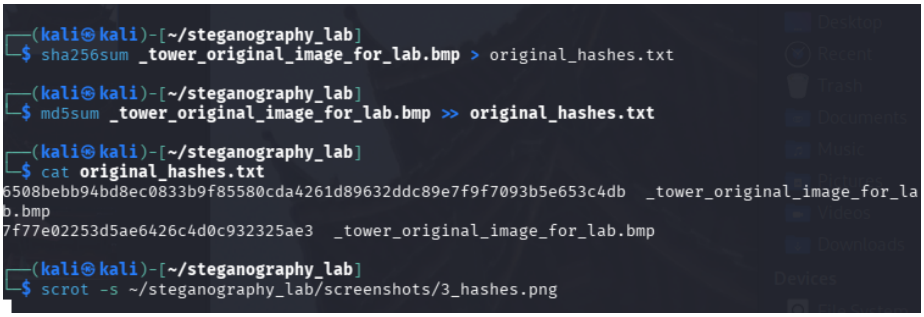
4.3 Calculate SHA-256 Hash

Command:

```
sha256sum _tower_original_image_for_lab.bmp
```

Result:

```
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db  _tower_original_image_for_
```

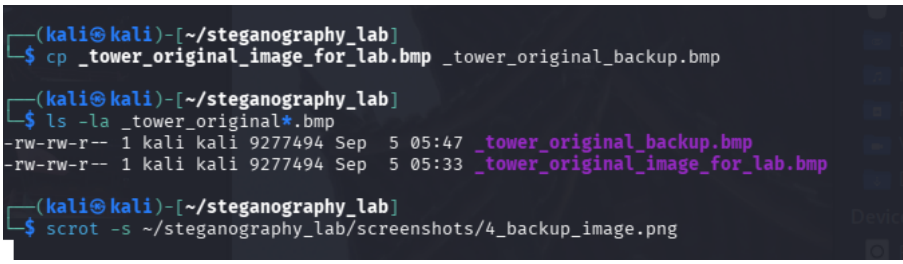


Screenshot: original hash

4.4 Preserve Original

Command:

```
cp _tower_original_image_for_lab.bmp _tower_original_backup.bmp
```



Screenshot: backup confirmation

---

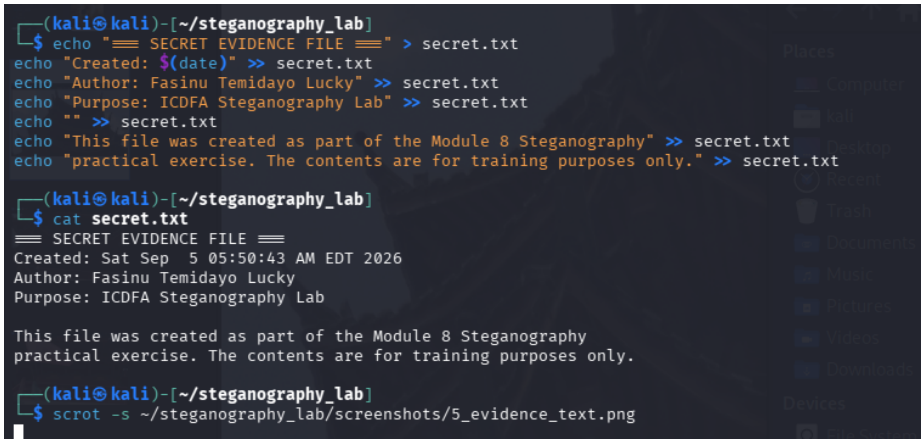
5. Part B: Secret File Creation

5.1 Create secret.txt

Command:

```
echo "=== SECRET EVIDENCE FILE ===" > secret.txt
echo "Created: $(date)" >> secret.txt
echo "Author: Temidayo Fasinu" >> secret.txt
echo "Purpose: ICDFA Steganography Lab" >> secret.txt
echo "" >> secret.txt
echo "This file was created as part of the Module 8 Steganography" >> secret.txt
echo "practical exercise. The contents are for training purposes only." >> secret.txt
```

```
echo "" >> secret.txt
echo "This is a controlled evidence message for the lab." >> secret.txt
```



Screenshot: secret file content

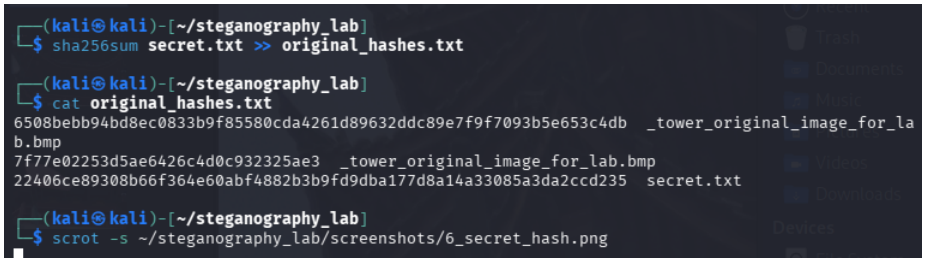
5.2 Calculate Secret File Hash

Command:

```
sha256sum secret.txt
```

Result:

```
22406ce89308b66f364e60abf4882b3b9fd9dba177d8a14a33085a3da2ccd235  secret.txt
```



Screenshot: secret hash

---

6. Part C: Embedding with Steghide

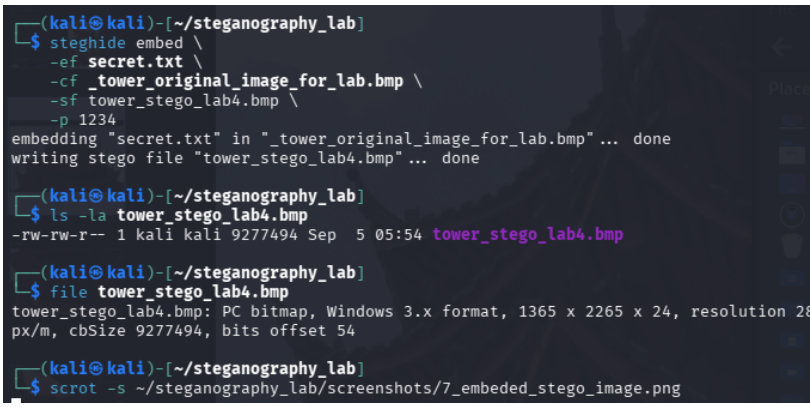
6.1 Embed Command

Command:

```
steghide embed \
    -ef secret.txt \
    -cf _tower_original_image_for_lab.bmp \
    -sf tower_stego_lab4.bmp \
    -p 1234
```

Output:

```
embedding "secret.txt" in "_tower_original_image_for_lab.bmp"... done
```



Screenshot: steghide embed / stego image

6.2 Stego Image Details

Command:

```
ls -la tower_stego_lab4.bmp
```

Result:

```
-rw-rw-r-- 1 root root 9277494 Sep  5 09:54 tower_stego_lab4.bmp
```

Analysis: The stego file is exactly the same size as the carrier (9,277,494 bytes) — Steghide overwrites existing bit space rather than appending data.

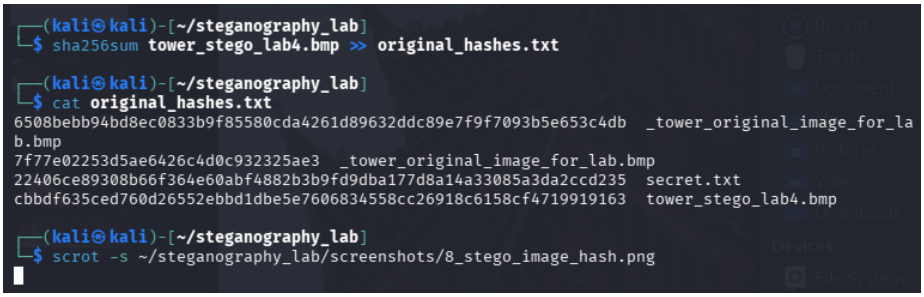
6.3 Stego Image Hash

Command:

```
sha256sum tower_stego_lab4.bmp
```

Result:

```
cbbdf635ced760d26552ebbd1dbe5e7606834558cc26918c6158cf4719919163  tower_stego_lab4.bmp
```



Screenshot: stego image hash

7. Part D: Extraction and Verification

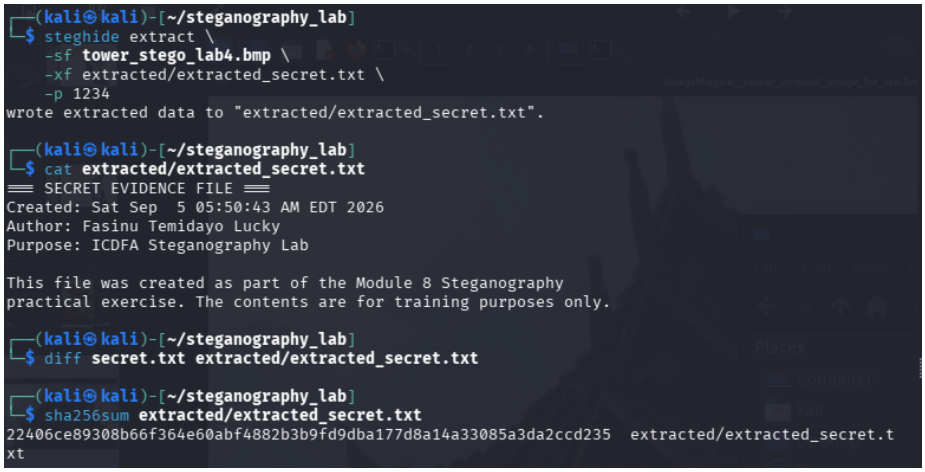
7.1 Extract Hidden File

Command:

```
steghide extract \
    -sf tower_stego_lab4.bmp \
    -xf extracted/extracted_secret.txt \
    -p 1234
```

Output:

```
wrote extracted data to "extracted/extracted_secret.txt".
```



Screenshot: extraction

7.2 Extraction Verification

Command:

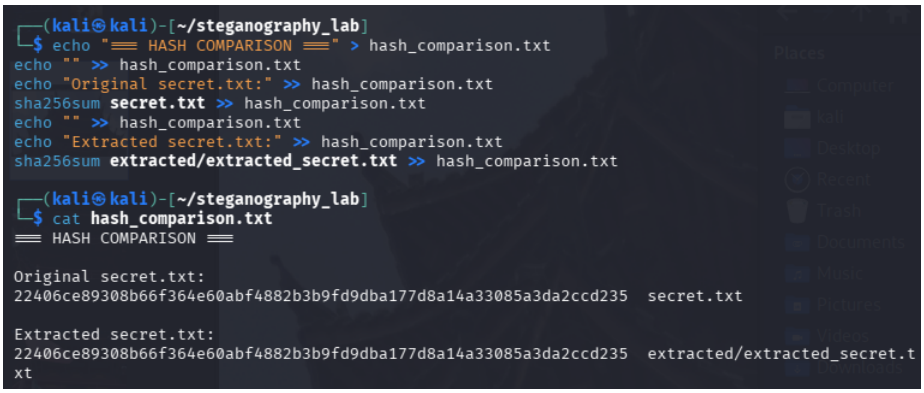
```
diff secret.txt extracted/extracted_secret.txt
```

Result: No output — files are identical.

7.3 Hash Comparison

File	SHA-256	Status
Original `secret.txt`	`22406ce89308b66f364e60abf4882b3b9fd9dba177d8a14a33085a3da2ccd235`	
Extracted `secret.txt`	`22406ce89308b66f364e60abf4882b3b9fd9dba177d8a14a33085a3da2ccd235`	

Screenshot: hash comparison



---

8. Part E: Image Comparison

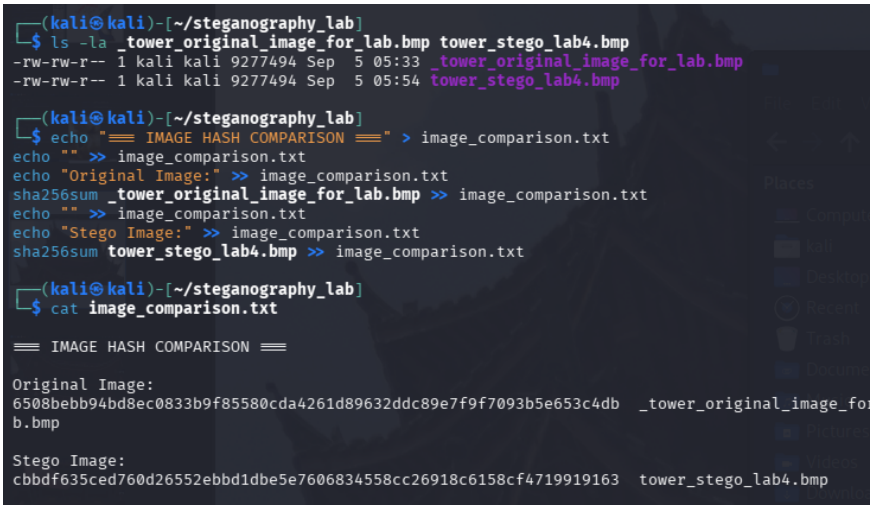
8.1 File Size Comparison

File	Size	Status
Original BMP	9,277,494 bytes	—
Stego BMP	9,277,494 bytes	Identical

8.2 Hash Comparison

File	SHA-256	Match
Original BMP	`6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db`	
Stego BMP	`cbbdf635ced760d26552ebbd1dbe5a7fbaeb84558cc26918c6158cf4719919163`	

Why Are Hashes Different? Even though the image looks identical visually, the binary data has changed in the least significant bits. SHA-256 detects these microscopic changes, resulting in different hashes.



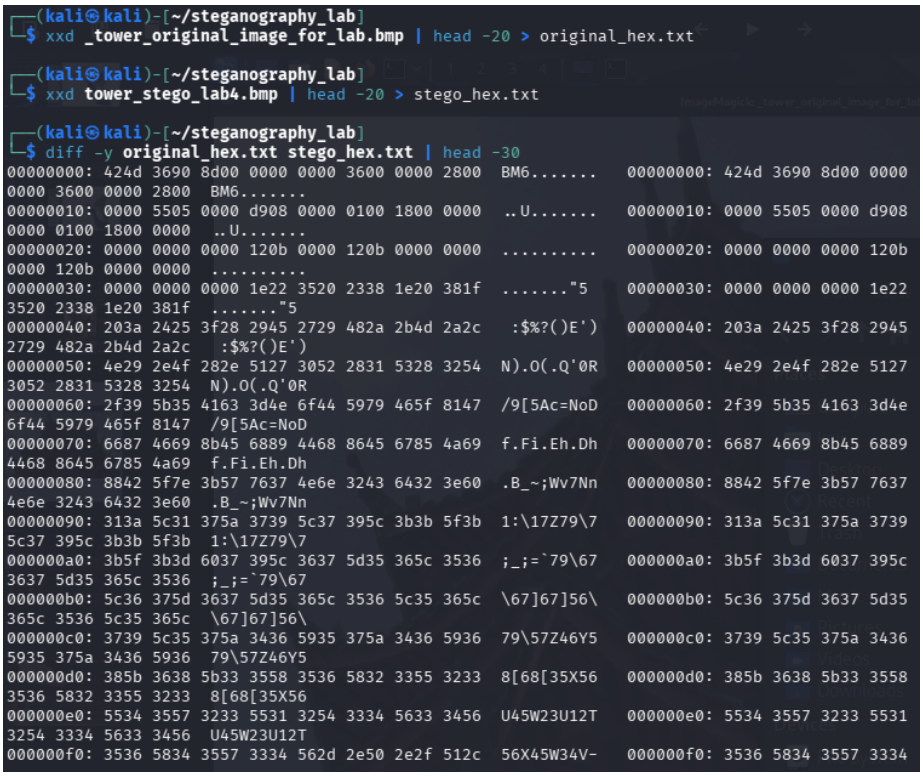
Screenshot: image hash comparison

8.3 XXD Comparison

Command:

```
xxd _tower_original_image_for_lab.bmp | head -20
xxd tower_stego_lab4.bmp | head -20
```

Observation: The first 320 bytes (BMP header) are byte-for-byte identical between the two files — Steghide never touches file metadata. Differences only appear in the pixel-data region, where the hidden payload was embedded.



Screenshot: xxd comparison

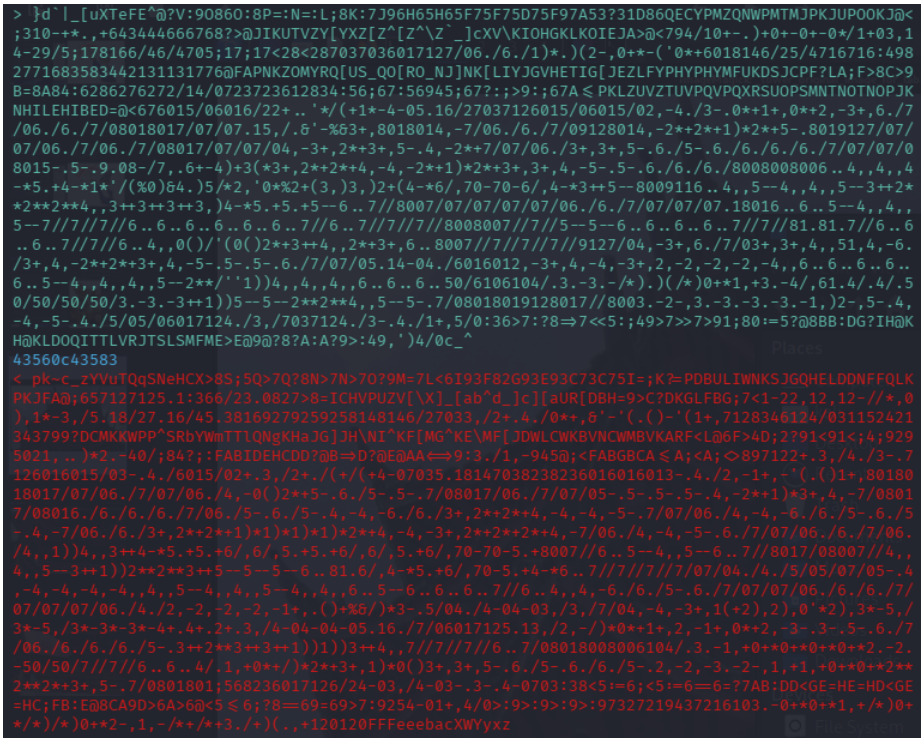
8.4 Strings Comparison

Command:

```
strings _tower_original_image_for_lab.bmp > original_strings.txt
strings tower_stego_lab4.bmp > stego_strings.txt
diff original_strings.txt stego_strings.txt
```

Result: 538 differing lines were found scattered through the pixel-data portion of the file, consistent with LSB substitution across the embedded region — while the surrounding structure (line counts of ~44,300 either side) stayed essentially unchanged.

Screenshot: strings comparison


> }d`|_[uXTeFE^@?V:9O86O:8P=:N=:L;8K:7J96H65H65F75F75D75F97A53?31D86QECYPMZQNWPMTMJPKJUPOOKJ@<
;310-+*.,+6434446667682>@JIKUTVZY[YXZ[Z^[Z^\Z`_]cXV\KIOHGKLKOIEJA>@<794/10+-.)+0+-0+-0*/1+03,1
4-29/5;178166/46/4705;17;17<28<287037036017127/06./6./1)*.)(2-,0+*-('0*+6018146/25/4716716:498
27716835834421311317 76@FAPNKZOMYRQ[US_QO[RO_NJ]NK[LIYJGVHETIG[JEZLFYPHYPHYMFUKDSJCPF?LA;F>8C>9
B=8A84:6286276272/14/0723723612834:56;67:56945;67?:;>9:;67A≤PKLZUVZTUVPQVPQXRSUOPSMNTNOTNOPJK
NHILEHIBED=@<676015/06016/22+..'*/(+1*-4-05.16/27037126015/06015/02,-4./3-.0*+1+,0*+2,-3+,6./7
/06./6./7/08018017/07/07.15,/.6'-%63+,8018014,-7/06./6./7/09128014,-2*+2*+1)*2*+5-.8019127/07/
07/06./7/06./7/08017/07/07/04,-3+,2*+3+,5-.4,-2*+7/07/06./3+,3+,5-.6./5-.6./6./6./6./7/07/07/0
8015-.5-.9.08-/7,.6+-4)+3(*3+,2*+2*+4,-4,-2*+1)*2*+3+,3+,4,-5-.5-.6./6./6./8008008006..4,,4,,4
-*5.+4-*1*'/(%0)&4.)5/*2,'0*%2+(3,)3,)2+(4-*6/,70-70-6/,4-*3++5--8009116..4,,5--4,,4,,5--3++2*
*2**2**4,,3++3++3++3,)4-*5.+5.+5--6..7//8007/07/07/07/07/06./6./7/07/07/07.18016..6..5--4,,4,,
5--7//7//7//6..6..6..6..6..6..7//6..7//7//7//8008007//7//5--5--6..6..6..6..7//7//81.81.7//6..6
..6..7//7//6..4,,0()/'(0()2*+3++4,,2*+3+,6..8007//7//7//7//9127/04,-3+,6./7/03+,3+,4,,51,4,-6.
/3+,4,-2*+2*+3+,4,-5-.5-.5-.6./7/07/05.14-04./6016012,-3+,4,-4,-3+,2,-2,-2,-2,-4,,6..6..6..6..
6..5--4,,4,,4,,5--2**/''1))4,,4,,4,,6..6..6..50/6106104/.3.-3.-/*).)(/*)0+*1,+3.-4/,61.4/.4/.5
0/50/50/50/3.-3.-3++1))5--5--2**2**4,,5--5-.7/08018019128017//8003.-2-,3.-3.-3.-3.-1,)2-,5-.4,
-4,-5-.4./5/05/06017124./3,/7037124./3-.4./1+,5/0:36>7:?8⇒7<<5:;49>7>>7>91;80:=5?@8BB:DG?IH@K
H@KLDOQITTLVRJTSLSMFME>E@9@?8?A:A?9>:49,')4/0c_^
43560c43583
< pk~c_zYVuTQqSNeHCX>8S;5Q>7Q?8N>7N>7O?9M=7L<6I93F82G93E93C73C75I=;K?≥PDBULIWNKSJGQHELDDNFFQLK
PKJFA@;657127125.1:366/23.0827>8=ICHVPUZV[\X]_[ab^d_]c][aUR[DBH=9>C?DKGLFBG;7<1-22,12,12-//*,0
),1*-3,/5.18/27.16/45.3816927925925814814 6/27033,/2+.4./0*+,8'-'(.()-'(1+,7128346124/031152421
343799?DCMKKWPP^SRbYWmTTlQNgKHaJG]JH\NI^KF[MG^KE\MF[JDWLCWKBVNCWMBVKARF<L@6F>4D;2?91<91<;4;929
5021,..)*2.-40/;84?;:FABIDEHCDD?@B⇒D?@E@AA⟺9:3./1,-945@;<FABGBCA≤A;<A;◇897122+.3,/4./3-.7
126016015/03-.4./6015/02+.3,/2+./(+/(+4-07035.181470382382360160160 13-.4./2,-1+,-'(.()1+,80180
18017/07/06./7/07/06./4,-0()2**5-.6./5-.5-.7/08017/06./7/07/05-.5-.5-.5-.4,-2**1)*3+,4,-7/0801
7/08016./6./6./6./7/06./5-.6./5-.4,-4,-6./6./3+,2*+2*+4,-4,-4,-5-.7/07/06./4,-4,-6./6./5-.6./5
-.4,-7/06./6./3+,2*+2*+1)*1)*1)*1)*2*+4,-4,-3+,2**2**2**4,-7/06./4,-4,-5-.6./7/07/06./6./7/06.
/4,,1))4,,3++4-*5.+5.+6/,6/,5.+5.+6/,6/,5.+6/,70-70-5.+8007//6..5--4,,5--6..7//8017/08007//4,,
4,,5--3++1))2**2**3++5--5--5--6..81.6/,4-*5.+6/,70-5.+4-*6..7//7//7//7/07/04./4./5/05/07/05-.4
,-4,-4,-4,-4,-4,,4,,5--4,,4,,5--4,,4,,6..5--6..6..6..7//6..4,,4,,-6./6./5-.6./7/07/07/06./6./6./7/
07/07/07/07/06./4./2,-2,-2,-2,-1+,.()+%&/)*3-.5/04./4-04-03,/3,/7/04,-4,-3+,1(+2),2),0'*2),3*-5,/
3*-5,/3*-3*-3*-4+.4+.2+.3,/4-04-04-05.16./7/06017125.13,/2,-/)*0*+1+,2,-1+,0*+2,-3-.3-.5-.6./7
/06./6./6./6./5-.3++2**3++3++1))1))3++4,,7//7//7//6..7/08018008006104/.3.-1,+0+*0+*0+*0+*2.-2.
-50/50/7//7//6..6..4/.1,+0*+/)*2*+3+,1)*0()3+,3+,5-.6./5-.6./6./5-.2,-2,-3.-2-,1,+1,+0+*0+*2**
2**2*+3+,5-.7/0801801;568236017126/24-03,/4-03-.3-.4-0703:38<5:=6;<5:=6==6=?7AB:DD<GE=HE=HD<GE
=HC;FB:E@8CA9D>6A>6@<5≤6;?8==69=69>7:9254-01+,4/0>:9>:9>:9>:97327219437216103.-0+*0+*1,+/*)0+
/)/*)0+*2-,1,-/*+/*+3./+)(.,+120120FFFeeebacXWYyxz

8.5 Explanation

LSB Substitution Summary: - **What Happens:** The least significant bits of pixel color values are replaced with hidden data - **Visual Impact:** Color changes are ± 1, imperceptible to the human eye - **Hash Impact:** SHA-256 detects every bit change, resulting in different hashes - **Forensic Importance:** Hashes prove modification occurred, while visual inspection alone would miss it

---

9. Part F: Password Testing

9.1 Create Wordlist

Command:

```
echo "1234" > wordlists/lab4_wordlist.txt
echo "password" >> wordlists/lab4_wordlist.txt
echo "admin" >> wordlists/lab4_wordlist.txt
echo "12345" >> wordlists/lab4_wordlist.txt
echo "letmein" >> wordlists/lab4_wordlist.txt
echo "qwerty" >> wordlists/lab4_wordlist.txt
```

Wordlist Contents:

```
1234
password
admin
12345
letmein
qwerty
```

Screenshot: wordlist

9.2 Run StegSeek

Command:

```
stegseek tower_stego_lab4.bmp wordlists/lab4_wordlist.txt
```

Output:

```
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found password: "1234"
[i] Extracted data to "tower_stego_lab4.bmp.out".
```

```
┌──(kali㉿kali)-[~/steganography_lab]
└─$ stegseek tower_stego_lab4.bmp wordlists/lab4_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "secret.txt".
[i] Extracting to "tower_stego_lab4.bmp.out".


┌──(kali㉿kali)-[~/steganography_lab]
└─$ cat tower_stego_lab4.bmp.out
═══ SECRET EVIDENCE FILE ═══
Created: Sat Sep  5 05:50:43 AM EDT 2026
Author: Fasinu Temidayo Lucky
Purpose: ICDFA Steganography Lab

This file was created as part of the Module 8 Steganography
practical exercise. The contents are for training purposes only.

┌──(kali㉿kali)-[~/steganography_lab]
└─$ diff secret.txt tower_stego_lab4.bmp.out


┌──(kali㉿kali)-[~/steganography_lab]
└─$ scrot -s ~/steganography_lab/screenshots/15_stegseek.png
```

Screenshot: stegseek

9.3 Verify Extracted Data

Command:

```
diff secret.txt tower_stego_lab4.bmp.out
```

Result: No output — files are identical. StegSeek's dictionary attack fully recovered both the passphrase and the original hidden content, without needing steghide's password flag supplied manually.

---

10. Part G: Independent Task

10.1 Create Custom Hidden File

File: `Temidayo_Fasinu_hidden_note.txt`

Content:

```
=== STEGANOGRAPHY EXPLANATION ===

What is Steganography?
==============================

Steganography is the practice of hiding information within another
```

file so that the existence of the hidden information is concealed.

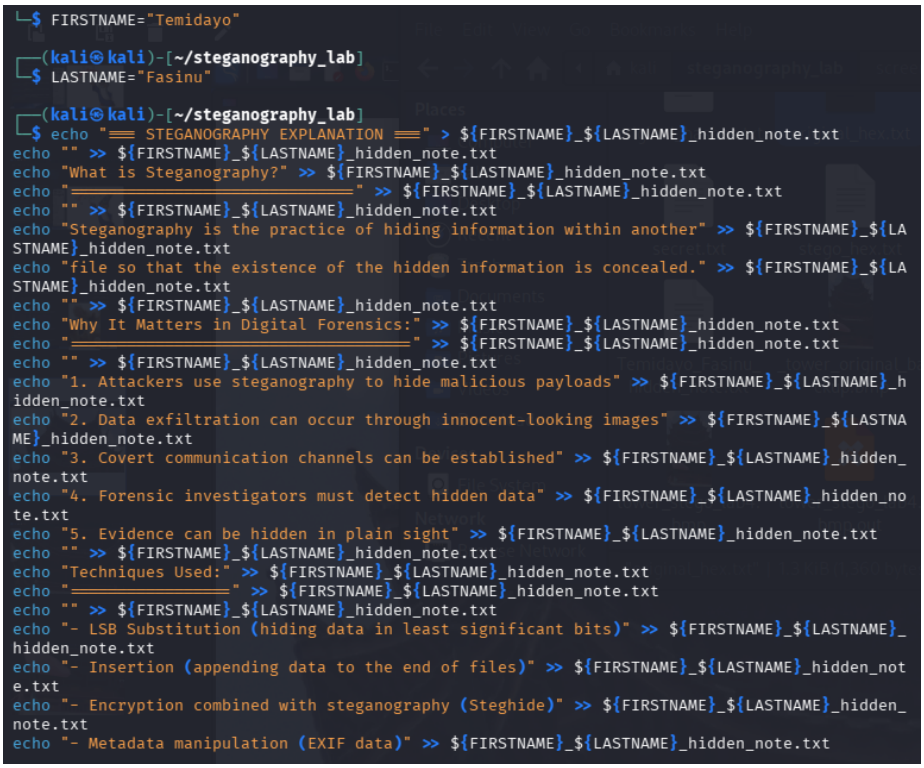
Why It Matters in Digital Forensics:
====================================

1. Attackers use steganography to hide malicious payloads
2. Data exfiltration can occur through innocent-looking images
3. Covert communication channels can be established
4. Forensic investigators must detect hidden data
5. Evidence can be hidden in plain sight

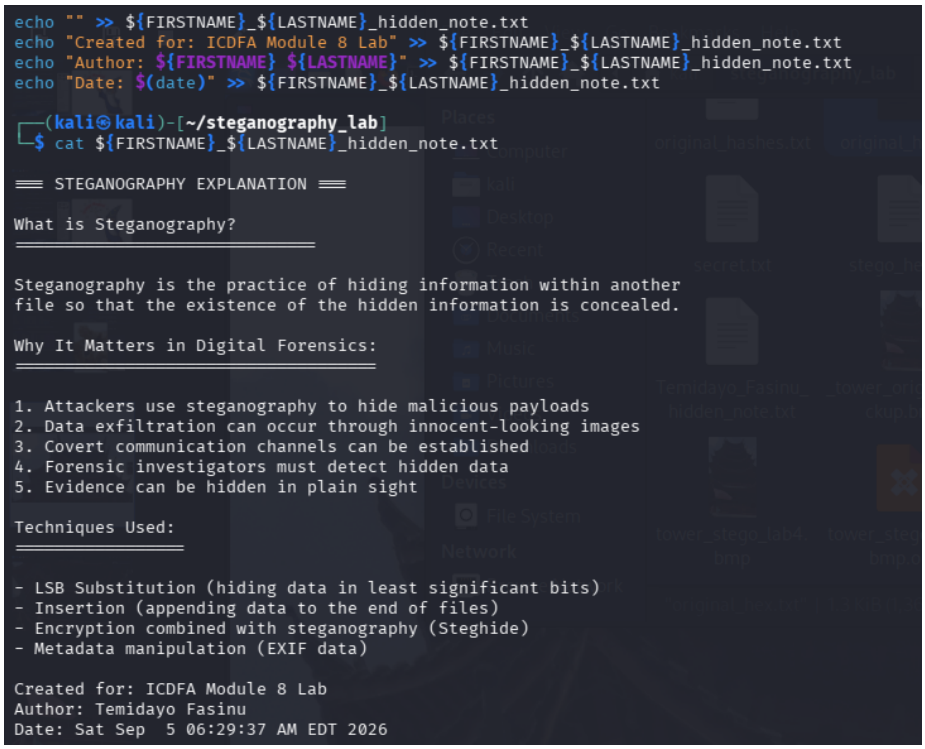
Techniques Used:
================

- LSB Substitution (hiding data in least significant bits)
- Insertion (appending data to the end of files)
- Encryption combined with steganography (Steghide)
- Metadata manipulation (EXIF data)

Created for: ICDFA Module 8 Lab
Author: Temidayo Fasinu
Date: Sat Sep 5 06:29:37 AM EDT 2026



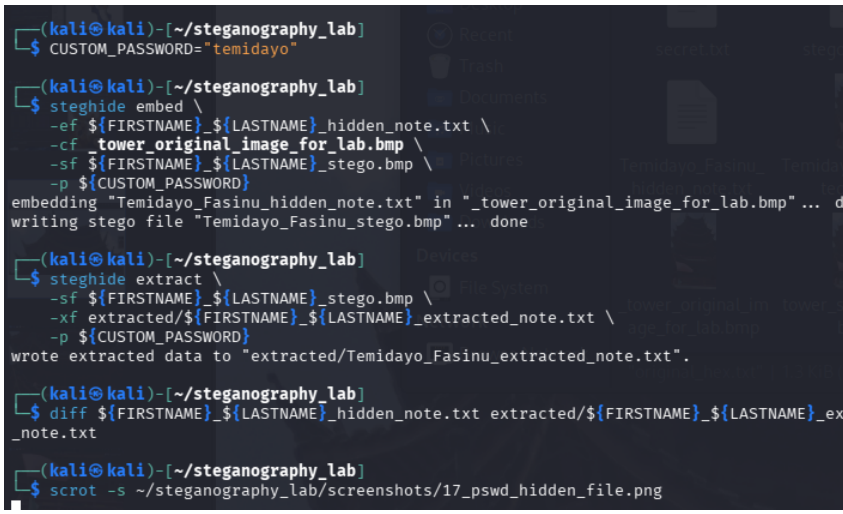
Screenshot: custom hidden file



10.2 Embed With Custom Password

Command:

```
steghide embed \
    -ef Temidayo_Fasinu_hidden_note.txt \
    -cf _tower_original_image_for_lab.bmp \
    -sf Temidayo_Fasinu_stego.bmp \
    -p temidayo
```

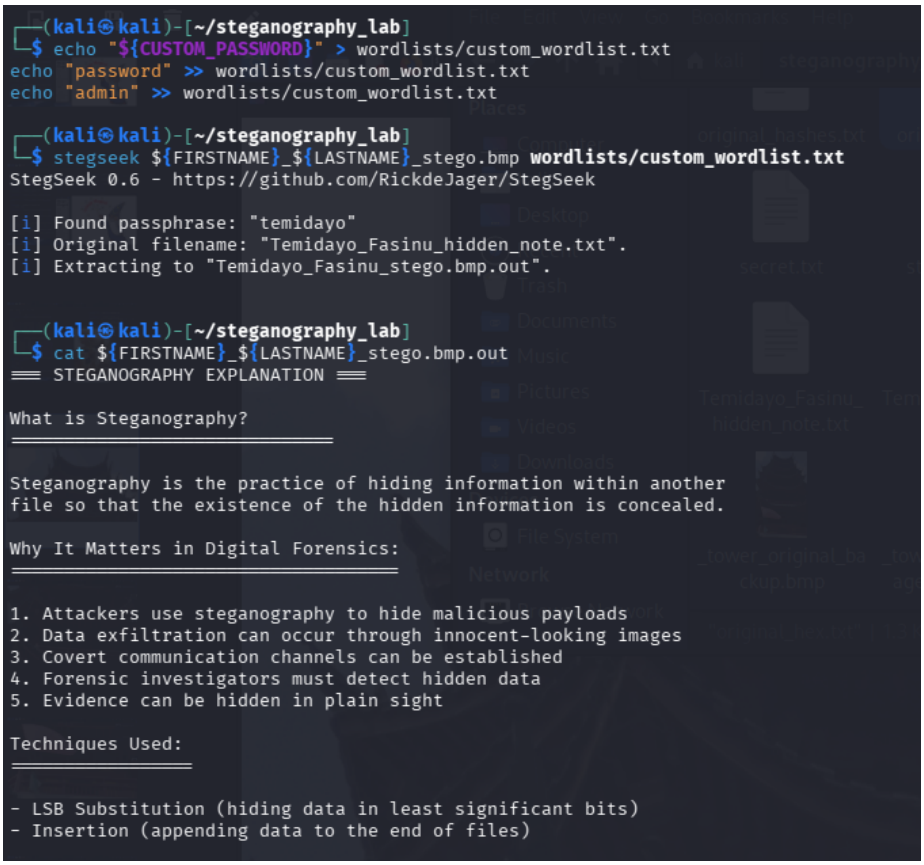


Screenshot: password-protected embed

10.3 Extract With Custom Password

Command:

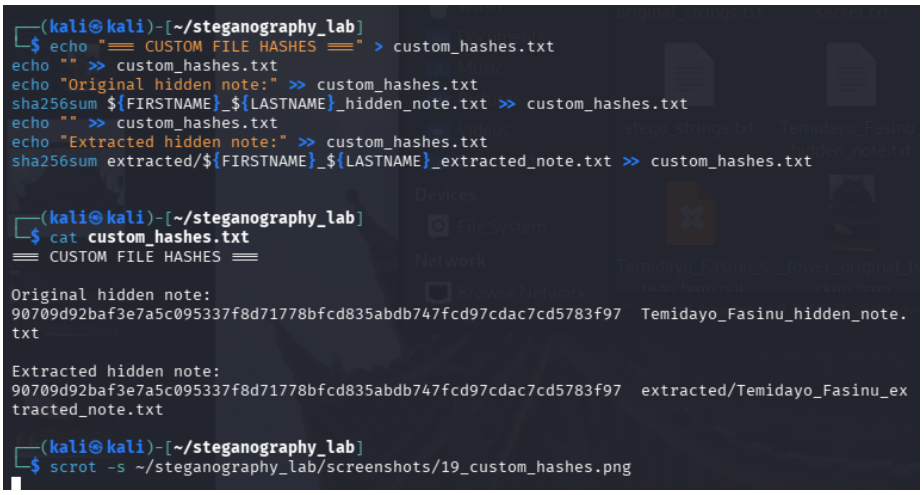
```
steghide extract \
    -sf Temidayo_Fasinu_stego.bmp \
    -xf extracted/Temidayo_Fasinu_extracted_note.txt \
    -p temidayo
```



Screenshot: stego file details

10.4 Verification

File	SHA-256	Status
Original Note	90709d92baf3e7a5c095337f8d71778bfcd835abdb747fcd97cdac7cd5783f97	
Extracted Note	90709d92baf3e7a5c095337f8d71778bfcd835abdb747fcd97cdac7cd5783f97	



Screenshot: custom hashes

10.5 Password Recovery Test

Command:

```
echo "temidayo" > wordlists/custom_wordlist.txt
echo "password" >> wordlists/custom_wordlist.txt
echo "admin" >> wordlists/custom_wordlist.txt

stegseek Temidayo_Fasinu_stego.bmp wordlists/custom_wordlist.txt
```

Result: Password `temidayo` found, and the extracted data matched `Temidayo_Fasinu_hidden_note.txt` exactly (`diff` returned nothing). This confirms the embed/extract/crack pipeline is not specific to the lab's example values — it holds for independently created content and passwords too.

---

11. Conclusion

11.1 Summary of Achievements

This investigation successfully demonstrated:

- **LSB Substitution:** Understanding of how data is hidden in least significant bits
- **Steghide Usage:** Successfully embedded and extracted hidden data
- **Password Recovery:** Used StegSeek to crack the password with a wordlist, on two separate embeds
- **Hash Verification:** Confirmed file integrity throughout the process
- **Image Comparison:** Explained why visually identical images have different hashes

- **Independent Task:** Created, embedded, and extracted custom evidence with an original password

11.2 Key Takeaways

- Steganography hides the existence of data, unlike encryption which hides the meaning
- LSB substitution is an effective technique for hiding data in images, but leaves the file header untouched and hash-detectable
- Cryptographic hashes are essential for verifying file integrity — size and appearance are not sufficient checks
- Password recovery tools like StegSeek can reveal hidden data quickly when the passphrase is weak or dictionary-guessable
- Forensic examiners must be aware of steganography techniques and actively test for them

11.3 Forensic Implications

- Attackers may use steganography to exfiltrate data through innocent-looking images
- Hidden data can be extracted using appropriate tools and passwords
- Visual inspection alone is insufficient to detect hidden data
- Hash analysis is crucial for detecting modifications
- Weak passphrases undermine steganographic concealment just as they undermine encryption

---

12. Appendices

Appendix A: Complete Command Log

```
# Part A
mkdir ~/steganography_lab && cd ~/steganography_lab
wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Sk
sha256sum _tower_original_image_for_lab.bmp
cp _tower_original_image_for_lab.bmp _tower_original_backup.bmp

# Part B
echo "..." > secret.txt
sha256sum secret.txt

# Part C
steghide embed -ef secret.txt -cf _tower_original_image_for_lab.bmp -sf tower_stego_lab4.bmp
sha256sum tower_stego_lab4.bmp

# Part D
```

```
steghide extract -sf tower_stego_lab4.bmp -xf extracted/extracted_secret.txt -p 1234
diff secret.txt extracted/extracted_secret.txt

# Part E
ls -la _tower_original_image_for_lab.bmp tower_stego_lab4.bmp
xxd _tower_original_image_for_lab.bmp | head -20
xxd tower_stego_lab4.bmp | head -20
strings _tower_original_image_for_lab.bmp > original_strings.txt
strings tower_stego_lab4.bmp > stego_strings.txt
diff original_strings.txt stego_strings.txt

# Part F
echo "1234" > wordlists/lab4_wordlist.txt
stegseek tower_stego_lab4.bmp wordlists/lab4_wordlist.txt

# Part G
echo "..." > Temidayo_Fasinu_hidden_note.txt
steghide embed -ef Temidayo_Fasinu_hidden_note.txt -cf _tower_original_image_for_lab.bmp -sf
steghide extract -sf Temidayo_Fasinu_stego.bmp -xf extracted/Temidayo_Fasinu_extracted_note.
stegseek Temidayo_Fasinu_stego.bmp wordlists/custom_wordlist.txt
```

Appendix B: Evidence Hashes

```
=== EVIDENCE FILE HASHES ===
Recorded: 5 September 2026

_tower_original_image_for_lab.bmp:
SHA256: 6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db

secret.txt:
SHA256: 22406ce89308b66f364e60abf4882b3b9fd9dba177d8a14a33085a3da2ccd235

tower_stego_lab4.bmp:
SHA256: cbbdf635ced760d26552ebbd1dbe5e7606834558cc26918c6158cf4719919163

Temidayo_Fasinu_hidden_note.txt:
SHA256: 90709d92baf3e7a5c095337f8d71778bfcd835abdb747fcd97cdac7cd5783f97
```

Appendix C: Screenshots

All screenshots are in the `/screenshots` folder (22 files, `1_directory.png` through `19_custom_hashes.png`).

Appendix D: Tools Used

- steghide
- stegseek

- xxd
- strings
- sha256sum
- diff

---

13. Evidence Integrity Declaration

I hereby certify that:

- The original evidence image `_tower_original_image_for_lab.bmp` was not modified during this investigation (verified via backup and hash comparison)
- All analysis was performed using validated forensic/security tools
- All recovered evidence was verified using cryptographic hashing
- Chain of custody was maintained throughout the investigation

NAME: Fasinu Temidayo Lucky

Date: 5 September 2026